



# 100+ SQL Interview Queries — The Complete Bank

💡 **How to use this:** Every query is self-contained with a one-line problem statement. Read the problem. Write the query yourself. Then check.  
Companies tested: Google, Amazon, Meta, Microsoft, Uber, Airbnb.

## 1. Basic SELECT & Filtering (01–10)

🎯 Every interview starts here. Get these perfect before anything else.

### 01 — Select all columns from a table

```
SELECT * FROM employees;
```

### 02 — Select specific columns with alias

```
SELECT first_name, last_name, salary AS monthly_salary  
FROM employees;
```

### 03 — Filter rows with WHERE

```
SELECT * FROM employees  
WHERE department = 'Sales' AND salary > 50000;
```

### 04 — Filter with OR and NOT

```
SELECT * FROM employees  
WHERE department = 'HR' OR department = 'Finance';  
  
SELECT * FROM employees  
WHERE NOT department = 'IT';
```

## 05 — Filter with IN and NOT IN

```
SELECT * FROM employees
WHERE department IN ('Sales', 'HR', 'Finance');

SELECT * FROM products
WHERE category NOT IN ('Electronics', 'Clothing');
```

## 06 — Filter with BETWEEN

```
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';

SELECT * FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

## 07 — Filter with LIKE (pattern matching)

```
SELECT * FROM customers WHERE name LIKE 'A%';      -- start
s with A
SELECT * FROM customers WHERE name LIKE '%son';     -- ends
with son
SELECT * FROM customers WHERE email LIKE '%@gmail%'; -- con
tains @gmail
```

## 08 — DISTINCT values

```
SELECT DISTINCT department FROM employees;
SELECT DISTINCT country, city FROM customers;
```

## 09 — ORDER BY single and multiple columns

```
SELECT * FROM employees ORDER BY salary DESC;
SELECT * FROM employees ORDER BY department ASC, salary DES
C;
```

## 10 — LIMIT and OFFSET

```
SELECT * FROM products ORDER BY price DESC LIMIT 10;
-- top 10
SELECT * FROM products ORDER BY price DESC LIMIT 10 OFFSET
10; -- page 2
```

## 2. Aggregation & GROUP BY (11–22)

🎯 Aggregation questions appear in every single data analyst interview.  
Master all HAVING patterns.

### 11 — COUNT, SUM, AVG, MIN, MAX

```
SELECT
    COUNT(*)                AS total_employees,
    SUM(salary)              AS total_payroll,
    ROUND(AVG(salary), 2)    AS avg_salary,
    MIN(salary)              AS lowest_salary,
    MAX(salary)              AS highest_salary
FROM employees;
```

### 12 — GROUP BY single column

```
SELECT department, COUNT(*) AS headcount, AVG(salary) AS av
g_salary
FROM employees
GROUP BY department
ORDER BY headcount DESC;
```

### 13 — GROUP BY multiple columns

```
SELECT department, job_title, COUNT(*) AS count, SUM(salar
y) AS total
FROM employees
GROUP BY department, job_title
ORDER BY department, total DESC;
```

### 14 — HAVING — filter after aggregation

```
SELECT department, COUNT(*) AS headcount
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

## 15 — HAVING with multiple conditions

```
SELECT department, COUNT(*) AS cnt, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING COUNT(*) > 3 AND AVG(salary) > 60000;
```

## 16 — WHERE and HAVING together

```
SELECT department, SUM(salary) AS total
FROM employees
WHERE status = 'active'           -- filter rows BEFORE group
                                  -- ing
GROUP BY department
HAVING SUM(salary) > 200000;      -- filter groups AFTER aggregation
```

## 17 — COUNT DISTINCT

```
SELECT
    COUNT(DISTINCT customer_id) AS unique_customers,
    COUNT(DISTINCT product_id) AS unique_products
FROM orders;
```

## 18 — Conditional aggregation with CASE WHEN

```
SELECT
    COUNT(*) AS total,
    SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_count,
    SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_count,
    ROUND(AVG(CASE WHEN department='Sales' THEN salary EN
```



```
D), 2) AS avg_sales_salary  
FROM employees;
```

## 19 — Pivot with CASE WHEN

```
SELECT  
    DATE_TRUNC('month', order_date) AS month,  
    SUM(CASE WHEN region='North' THEN revenue ELSE 0 END) AS  
S north,  
    SUM(CASE WHEN region='South' THEN revenue ELSE 0 END) AS  
S south,  
    SUM(CASE WHEN region='East' THEN revenue ELSE 0 END) AS  
S east,  
    SUM(revenue)  
AS total  
FROM orders  
GROUP BY 1 ORDER BY 1;
```

## 20 — Find departments where avg salary > company average

```
SELECT department, ROUND(AVG(salary), 2) AS dept_avg  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees)  
ORDER BY dept_avg DESC;
```

## 21 — Top N per group using aggregation

```
-- Products with more than 100 total orders  
SELECT product_id, COUNT(*) AS order_count  
FROM order_items  
GROUP BY product_id  
HAVING COUNT(*) > 100  
ORDER BY order_count DESC;
```

## 22 — ROLLUP — subtotals and grand total

```
SELECT department, job_title, SUM(salary) AS total
FROM employees
GROUP BY ROLLUP(department, job_title)
ORDER BY department, job_title;
```

### 3. JOINS (23–34)

🎯 JOIN questions separate junior from senior candidates. Know all types and the anti-join pattern cold.

#### 23 — INNER JOIN

```
SELECT o.order_id, c.customer_name, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

#### 24 — LEFT JOIN — all left rows

```
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

#### 25 — Anti-join — customers with NO orders

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

#### 26 — FULL OUTER JOIN

```
SELECT c.customer_name, o.order_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

#### 27 — Self JOIN — employee and manager

```

SELECT
    e.name AS employee,
    e.salary AS emp_salary,
    m.name AS manager,
    m.salary AS mgr_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id;

```

## 28 — Employees earning more than their manager

```

SELECT e.name AS employee, e.salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;

```

## 29 — Multi-table JOIN (4 tables)

```

SELECT
    o.order_id, c.customer_name, p.product_name,
    cat.category_name, o.quantity, o.revenue
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p ON o.product_id = p.product_id
LEFT JOIN categories cat ON p.category_id = cat.category_id
WHERE o.order_date >= '2024-01-01';

```

## 30 — Non-equi JOIN — salary bands

```

SELECT e.name, e.salary, b.band_name
FROM employees e
JOIN salary_bands b
    ON e.salary BETWEEN b.min_salary AND b.max_salary;

```

## 31 — CROSS JOIN — all combinations

```

SELECT p.product_name, r.region_name
FROM products p

```

```
CROSS JOIN regions r;
```

### 32 — Find products bought by BOTH customer A and B

```
SELECT DISTINCT o1.product_id
FROM orders o1
JOIN orders o2 ON o1.product_id = o2.product_id
WHERE o1.customer_id = 'A'
      AND o2.customer_id = 'B';
```

### 33 — Customers who bought A but NOT B

```
SELECT DISTINCT customer_id FROM orders WHERE product_id =
'A'
EXCEPT
SELECT DISTINCT customer_id FROM orders WHERE product_id =
'B';
```

### 34 — Category revenue share with JOIN

```
SELECT
    c.category_name,
    SUM(o.revenue)
AS revenue,
    ROUND(SUM(o.revenue)*100.0 / SUM(SUM(o.revenue))) OVER
(,2) AS pct_of_total
FROM orders o
JOIN products p  ON o.product_id  = p.product_id
JOIN categories c ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY revenue DESC;
```

## 4. Subqueries & CTEs (35–46)

 CTEs make complex queries readable. Subqueries test logical thinking.  
Both appear in every senior interview.

### 35 — Subquery in WHERE



```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

### 36 — Subquery in FROM (derived table)

```
SELECT department, avg_sal
FROM (
    SELECT department, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department
) dept_stats
WHERE avg_sal > 70000;
```

### 37 — Correlated subquery

```
-- Employees who earn more than their department average
SELECT name, department, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = e.department
);
```

### 38 — EXISTS

```
-- Customers who have placed at least one order
SELECT customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

### 39 — NOT EXISTS (anti-join)

```
-- Customers who have NEVER ordered
SELECT customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

#### 40 — CTE — basic

```
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT e.name, e.salary, d.avg_salary,
    ROUND(e.salary - d.avg_salary, 2) AS diff_from_avg
FROM employees e
JOIN dept_avg d ON e.department = d.department;
```

#### 41 — Multiple CTEs

```
WITH
monthly_rev AS (
    SELECT DATE_TRUNC('month', order_date) AS month, SUM(revenue) AS revenue
    FROM orders GROUP BY 1
),
ranked AS (
    SELECT *, RANK() OVER (ORDER BY revenue DESC) AS rnk
    FROM monthly_rev
)
SELECT month, revenue, rnk
FROM ranked
WHERE rnk <= 3;
```

#### 42 — Recursive CTE — org hierarchy

```

WITH RECURSIVE hierarchy AS (
    SELECT employee_id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.name, e.manager_id, h.level + 1
    FROM employees e
    JOIN hierarchy h ON e.manager_id = h.employee_id
)
SELECT level, name FROM hierarchy ORDER BY level, name;

```

#### 43 — CTE for deduplication

```

WITH deduped AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
        updated_at DESC) AS rn
    FROM customers
)
SELECT * FROM deduped WHERE rn = 1;

```

#### 44 — IN with subquery

```

SELECT * FROM products
WHERE category_id IN (
    SELECT category_id FROM categories WHERE parent_category = 'Electronics'
);

```

#### 45 — Subquery to find Nth highest salary

```

-- 3rd highest salary
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;

-- Using subquery (works in all dialects)
SELECT MIN(salary) FROM employees

```

```
WHERE salary IN (  
    SELECT DISTINCT salary FROM employees ORDER BY salary D  
    ESC LIMIT 3  
);
```

#### 46 — CTE with running total

```
WITH daily AS (  
    SELECT order_date, SUM(revenue) AS daily_rev  
    FROM orders GROUP BY order_date  
)  
SELECT  
    order_date, daily_rev,  
    SUM(daily_rev) OVER (ORDER BY order_date) AS running_to  
tal  
FROM daily;
```

## 5. Window Functions (47–60)

 Window functions are the most tested advanced topic. Every senior data role tests these heavily.

#### 47 — ROW\_NUMBER

```
SELECT name, department, salary,  
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY sal  
ary DESC) AS row_num  
FROM employees;
```

#### 48 — Top 1 per group (ROW\_NUMBER)

```
WITH ranked AS (  
    SELECT *,  
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY  
salary DESC) AS rn  
    FROM employees
```



```
)  
SELECT * FROM ranked WHERE rn = 1;
```

#### 49 — RANK vs DENSE\_RANK

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS rank_with_g  
aps,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS rank_no_gap  
s  
FROM employees;
```

#### 50 — NTILE — quartile bucketing

```
SELECT customer_id, ltv,  
       NTILE(4) OVER (ORDER BY ltv) AS quartile,  
       CASE NTILE(4) OVER (ORDER BY ltv)  
         WHEN 4 THEN 'Premium' WHEN 3 THEN 'High'  
         WHEN 2 THEN 'Mid'      ELSE 'Low'  
       END AS segment  
FROM customer_ltv;
```

#### 51 — LAG — previous row value

```
SELECT order_date, revenue,  
       LAG(revenue) OVER (ORDER BY order_date) AS prev_revenu  
e,  
       revenue - LAG(revenue) OVER (ORDER BY order_date) AS ch  
ange  
FROM daily_sales;
```

#### 52 — LEAD — next row value

```
SELECT order_date, revenue,  
       LEAD(revenue) OVER (ORDER BY order_date) AS next_revenu  
e  
FROM daily_sales;
```

### 53 — Month-over-Month % growth

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date) AS month, SUM(revenue) AS rev  
    FROM orders GROUP BY 1  
)  
SELECT month, rev,  
    ROUND((rev - LAG(rev) OVER (ORDER BY month)) * 100.0  
        / NULLIF(LAG(rev) OVER (ORDER BY month), 0), 2) AS  
mom_pct  
FROM monthly;
```

### 54 — Running total

```
SELECT order_date, revenue,  
    SUM(revenue) OVER (  
        ORDER BY order_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS running_total  
FROM daily_sales;
```

### 55 — Rolling 7-day average

```
SELECT order_date, revenue,  
    ROUND(AVG(revenue) OVER (  
        ORDER BY order_date  
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
    ), 2) AS rolling_7d_avg  
FROM daily_sales;
```

### 56 — FIRST\_VALUE and LAST\_VALUE

```
SELECT name, department, salary,  
    FIRST_VALUE(salary) OVER (PARTITION BY department ORDER  
BY salary DESC) AS top_salary,  
    LAST_VALUE(salary) OVER (  
        PARTITION BY department ORDER BY salary DESC
```

```

        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS bottom_salary
FROM employees;

```

## 57 — PERCENT\_RANK and CUME\_DIST

```

SELECT name, salary,
       ROUND(PERCENT_RANK() OVER (ORDER BY salary) * 100, 1) AS percentile_rank,
       ROUND(CUME_DIST() OVER (ORDER BY salary) * 100, 1) AS cumulative_pct
FROM employees;

```

## 58 — % of total per partition

```

SELECT
    department, name, salary,
    ROUND(salary * 100.0 / SUM(salary) OVER (PARTITION BY department), 2) AS pct_of_dept
FROM employees;

```

## 59 — Session detection

```

WITH gaps AS (
    SELECT *,
           event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) AS gap
    FROM events
),
flagged AS (
    SELECT *, CASE WHEN gap > INTERVAL '30 minutes' OR gap IS NULL THEN 1 ELSE 0 END AS new_session
    FROM gaps
)
SELECT *, SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
FROM flagged;

```

## 60 — Consecutive days detection

```
WITH daily AS (  
    SELECT DISTINCT user_id, DATE(login_time) AS login_date  
FROM logins  
)  
grp AS (  
    SELECT user_id, login_date,  
        login_date - ROW_NUMBER() OVER (PARTITION BY user_id  
ORDER BY login_date)::int AS grp  
    FROM daily  
)  
SELECT user_id, MIN(login_date) AS start_date,  
    MAX(login_date) AS end_date, COUNT(*) AS streak  
FROM grp  
GROUP BY user_id, grp  
HAVING COUNT(*) >= 3  
ORDER BY streak DESC;
```

## ♥ 6. NULL Handling (61–68)

🎯 NULL behaviour trips up even experienced developers. These are high-frequency interview traps.

### 61 — IS NULL and IS NOT NULL

```
SELECT * FROM employees WHERE manager_id IS NULL;      -- top-level managers  
SELECT * FROM employees WHERE manager_id IS NOT NULL;  -- everyone else
```

### 62 — COALESCE — first non-null value

```
SELECT  
    name,  
    COALESCE(phone, mobile, email, 'No contact') AS contact_info,  
    COALESCE(bonus, 0) + salary AS total_c
```



```
omp  
FROM employees;
```

### 63 — NULLIF — convert value to NULL

```
-- Prevents divide by zero  
SELECT revenue / NULLIF(orders, 0) AS avg_order_value  
FROM sales;  
  
-- Convert empty string to NULL  
SELECT NULLIF(TRIM(phone), '') AS phone  
FROM customers;
```

### 64 — NULL in aggregations

```
-- COUNT(*) counts all rows including NULLs  
-- COUNT(column) skips NULLs  
SELECT  
    COUNT(*) AS total_rows,  
    COUNT(salary) AS rows_with_salary, -- skips NULLs  
    COUNT(*) - COUNT(salary) AS null_count  
FROM employees;
```

### 65 — NULL in comparisons (the trap)

```
-- NULL = NULL is UNKNOWN, not TRUE  
-- These return NO rows:  
SELECT * FROM employees WHERE manager_id = NULL; -- WRONG  
SELECT * FROM employees WHERE NULL = NULL; -- WRON  
G  
  
-- Correct:  
SELECT * FROM employees WHERE manager_id IS NULL; -- CORRE  
CT
```

### 66 — ISNULL / IFNULL / NVL (dialect variants)

```
-- PostgreSQL / Standard SQL
SELECT COALESCE(bonus, 0) AS bonus FROM employees;

-- MySQL
SELECT IFNULL(bonus, 0) AS bonus FROM employees;

-- SQL Server
SELECT ISNULL(bonus, 0) AS bonus FROM employees;

-- Oracle
SELECT NVL(bonus, 0) AS bonus FROM employees;
```

## 67 — CASE WHEN for NULL replacement

```
SELECT name,
       CASE WHEN manager_id IS NULL THEN 'CEO / Top Level'
            ELSE 'Has Manager'
       END AS level
FROM employees;
```

## 68 — NULL-safe comparison

```
-- Compare two columns that may both be NULL
SELECT *
FROM table1 t1
JOIN table2 t2
  ON (t1.col = t2.col OR (t1.col IS NULL AND t2.col IS NU
LL));
```

# 7. String & Date Functions (69–80)

🎯 String and date manipulation separates analysts who write real queries from those who only know SELECT.

## 69 — String functions

```

SELECT
    UPPER(name)                AS upper_name,
    LOWER(email)               AS lower_email,
    LENGTH(name)               AS name_length,
    TRIM('  Alice  ')          AS trimmed,
    SUBSTRING(name, 1, 3)       AS first_3,
    CONCAT(first_name, ' ', last_name) AS full_name,
    REPLACE(phone, '-', '')     AS clean_phone,
    POSITION('@' IN email)       AS at_position
FROM employees;

```

## 70 — Extract parts of a string

```

SELECT
    email,
    SPLIT_PART(email, '@', 1) AS username,    -- PostgreSQL
    SPLIT_PART(email, '@', 2) AS domain
FROM customers;

-- MySQL equivalent
SELECT SUBSTRING_INDEX(email, '@', 1) AS username,
       SUBSTRING_INDEX(email, '@', -1) AS domain
FROM customers;

```

## 71 — Date functions

```

SELECT
    CURRENT_DATE                AS today,
    CURRENT_TIMESTAMP            AS now,
    EXTRACT(YEAR FROM order_date) AS year,
    EXTRACT(MONTH FROM order_date) AS month,
    EXTRACT(DOW FROM order_date) AS day_of_week,
    DATE_TRUNC('month', order_date) AS month_start,
    DATE_TRUNC('year', order_date) AS year_start
FROM orders;

```

## 72 — Date arithmetic